

Learning STARK

Safe Systems Programming for Typed ML Deployment

CHAPTER 01

A Language for Safer Deployment

Learning STARK

Chapter 1 preview: *A Language for Safer Deployment*

Early Access manuscript - July 2026

This chapter documents a language specification under active implementation. Core v1 and tensor extension v0.1 are normative drafts and have not yet been validated by a conforming compiler. Illustrative commands and diagnostics are identified in the text.

The design and text are original to the STARK Language Project. Product and project names mentioned for interoperability remain the property of their respective owners.

Source manuscript: `book/manuscript/ch01-safer-deployment.md`

A Language for Safer Deployment

The most expensive deployment defect is often not a sophisticated numerical failure. It is an ordinary disagreement about what a value is.

Software does not become reliable merely because its model is accurate. Between a request arriving at a service and a prediction leaving it, a deployment program decodes bytes, validates fields, changes image layout, converts element types, normalizes values, assembles batches, selects a device, invokes a model, and interprets the result. Every one of those steps carries assumptions. In many production stacks, those assumptions live in comments, variable names, configuration files, test fixtures, and the memories of engineers.

STARK begins with a different premise: when an important assumption can be expressed as part of a program's type, the compiler should be allowed to check it before the program runs.

That premise sounds modest. It is not. Applying it across an inference pipeline changes where errors are discovered, how interfaces are reviewed, and what a successful build means. A tensor ceases to be merely "some array." It can carry an element type, a rank, a symbolic shape, and a device. A model ceases to be merely a path passed to a runtime. Its ports can form a checked contract. Ownership ceases to be an informal convention about which component may mutate or release a value. It becomes part of the language.

This chapter introduces that design argument. It explains the deployment problem STARK targets, the guarantees supplied by Core v1, the additional guarantees proposed by the tensor extension, and the boundary between compile-time knowledge and runtime evidence. It also states the project's limitations plainly. STARK is currently a specification and early compiler

scaffold, not a production toolchain. The language must still earn its claims through implementation.

Project status

Core v1 and tensor extension v0.1 are normative drafts. No conforming STARK compiler, standard library, package manager, ONNX importer, or production runtime exists yet. Listings in this chapter describe specified or planned behavior and are labelled accordingly.

1.1 The program around the model

Discussions about machine-learning deployment often focus on the model executor: ONNX Runtime, TensorRT, IREE, a framework runtime, or a vendor accelerator. That focus is understandable. The model contains the expensive operations, and the backend determines much of the system's throughput and hardware support.

But the backend does not own the entire program. Real services contain substantial code before and after model execution. Consider a computer-vision endpoint receiving an encoded image:

- 1.** The request is authenticated and decoded.
- 2.** The payload is checked for size and content type.
- 3.** Image bytes become pixels with a height, width, channel count, and element representation.
- 4.** Pixels are resized or cropped.
- 5.** Channel order changes from the decoder's convention to the model's convention.
- 6.** Integer samples become floating-point values.
- 7.** Values are scaled and normalized.
- 8.** One image becomes a batch, or joins an existing batch.
- 9.** Data is transferred to the device on which the model executes.
- 10.** The model is called with one or more inputs.
- 11.** Outputs are reduced, decoded, filtered, or mapped to domain objects.
- 12.** A response is serialized and returned.

The model runtime may validate its immediate inputs, but it cannot reconstruct the intent of every earlier transformation. A rank-four floating-point tensor is compatible with many interpretations. It could represent images in NCHW order, images in NHWC order, feature maps, video frames, or unrelated data that happens to have four axes. Two arrays can have the same number of elements and still describe different things.

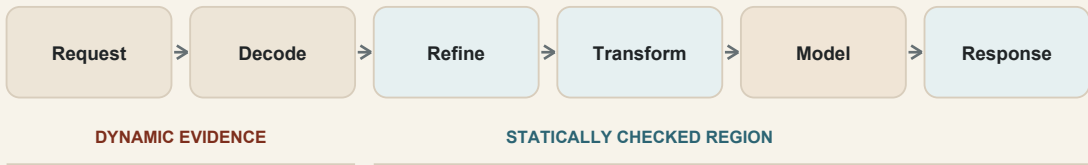


Figure 1-1. The checked program surrounds, rather than replaces, the model backend.

This is the program around the model: the ordinary deployment code whose job is to turn untrusted, dynamic input into a value that satisfies a model's contract, then turn a model's output into something useful. STARK's initial product wedge is not to replace the numerical backend. It is to make this surrounding program more explicit and more checkable.

That distinction matters. A new tensor runtime would compete on kernels, graph optimization, accelerator coverage, memory planning, quantization, and numerical fidelity. Those are enormous fields with mature implementations. STARK instead asks whether a language front end can eliminate a useful class of integration defects while delegating numerical execution to existing systems.

The question is intentionally narrow:

Can a language catch shape, element-type, device, ownership, and model-signature defects before inference begins, without requiring a new ML runtime?

The project roadmap is organized to produce evidence for or against that proposition. If the answer is no, the design should be revised or stopped. If the same guarantees can be delivered more simply by a schema generator or library, building a new general-purpose language would be difficult to justify.

1.2 Why deployment defects survive ordinary code review

Many deployment mistakes look obvious after they are found. A model expects three channels but receives four. A preprocessing function produces NHWC data while the model expects NCHW. A normalization constant has the wrong shape. A tensor remains on the CPU while another operand is on a GPU. An artifact is updated but a handwritten interface is not.

The difficulty is not that engineers cannot understand these rules. The difficulty is that the rules are distributed across layers.

A decoder knows the pixel layout it produces. A preprocessing function knows the layout it expects. A model artifact records some of its input contract. Host code selects the device. A configuration file selects the artifact. A test may exercise only one batch size. A variable named `images` communicates intent to a human but nothing enforceable to a compiler. Each local fragment can look reasonable while the assembled pipeline remains inconsistent.

Dynamic languages intensify this effect because a value may carry less static information than the engineer knows conceptually. A type annotation such as `ndarray` or `Tensor` says that a value is array-like, but often not that it contains unsigned bytes in `[B, H, W, 3]`, or floating-point samples in `[B, 3, 224, 224]`, or that it resides on a particular device. External shape-annotation tools can help, but they are rarely the single authority across request handling, transformations, model declarations, and execution.

Static typing is not automatically sufficient either. A statically typed host language can faithfully represent a runtime tensor handle while leaving its shape and device dynamic. The host compiler then proves that the handle is used according to the runtime API, not that the model call is semantically compatible with the preceding pipeline.

Tests remain essential, but tests sample executions. They are strongest when checking behavior that depends on data: numerical tolerances, unusual images, empty results, backend failures, and performance. They are a less satisfying sole defense for structural contracts that apply to every call. If a function is valid only when its inner dimensions match, or a model always requires `Float32` in `[B, 3, 224, 224]`, the contract is more naturally expressed once and checked at every call site.

Design principle: move stable facts into the program

Use runtime validation for facts that arrive from the outside world. Once validated, preserve those facts in types so later operations do not repeatedly rediscover them.

The aim is not to abolish runtime errors. Files can be missing. Requests can be malformed. Devices can run out of memory. Backends can fail. Networks can disconnect. The aim is to separate two categories:

- **Evidence-dependent failures**, which require inspecting runtime data or external state.
- **Program contradictions**, which follow from declarations already available to the compiler.

STARK tries to reject the second category during compilation and make the first category explicit through `Result` and carefully placed refinement operations.

1.3 Five concerns at the deployment boundary

The active roadmap uses five concerns as a review lens: **shape**, **meaning**, **location**, **ownership**, and **constraints**. They are not necessarily five independent parameters on one universal tensor type. They are five questions that reveal where a deployment interface may be underspecified.

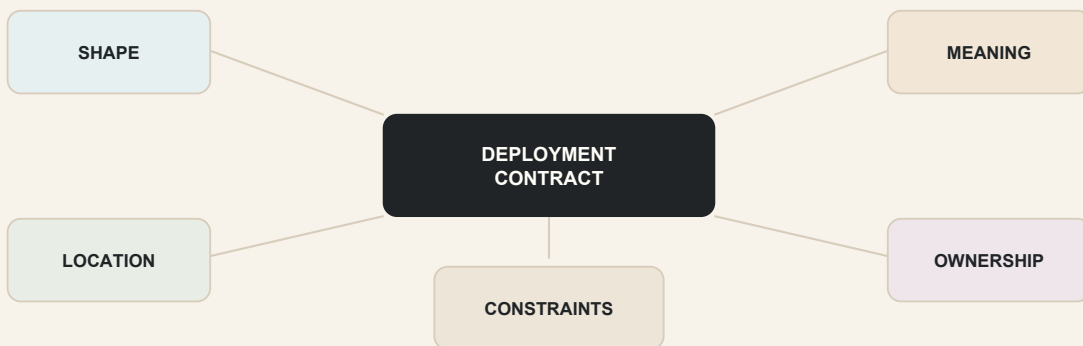


Figure 1-2. Five questions expose underspecified deployment interfaces.

Shape

Shape includes rank, axis sizes, and relationships between dimensions. A model might accept a batch of RGB images with type `[B, 3, 224, 224]`, where `B` is symbolic. A matrix multiplication may accept `[M, K]` and `[K, N]` and return `[M, N]`. The shared `K` is not documentation; it is a constraint.

Some dimensions are static literals. Others vary from one call to another but must remain consistent within a call. Treating both as unstructured integers loses useful information. The tensor extension therefore distinguishes literal dimensions from symbolic dimension parameters and defines a deliberately limited equality system for dimension expressions.

The limitation is important. Tensor extension v0.1 normalizes polynomial expressions and checks equality of their normal forms. It does not attempt arbitrary theorem proving or general range analysis. A compiler may know that $B * (H + 1)$ equals $B * H + B$; it does not therefore know every inequality involving `B` and `H`. Rejecting an unprovable operation is preferable to silently introducing a runtime assumption.

Meaning

Meaning describes what axes and values represent: batch, channel, height, width, color space, value range, coordinate frame, token position, or class score. Two tensors can be identical in shape and element type while differing in meaning.

Core tensor extension v0.1 does not attempt to solve this entire problem. Named axes, color spaces, value ranges, and coordinate frames are reserved for later experiments. The roadmap explicitly delays semantic CV annotations until the basic shape, dtype, device, and model-signature pipeline has been implemented and evaluated.

This restraint prevents the type system from claiming more than it can support. A shape checker can prove that two inner dimensions agree. It cannot infer that a three-element axis represents RGB rather than BGR unless the language or a library gives that distinction a formal representation. For now, domain types such as `Image` should begin as library-level abstractions, with tensor types providing the lower-level structural contract.

Location

Location is where a value resides and where an operation will execute. In tensor extension v0.1, device placement may be implicit and polymorphic, or fixed explicitly with a device such as `Cpu` or `Cuda<0>`. Multi-operand operations require devices to unify. Transfers are explicit.

This prevents a common ambiguity: whether an operation will copy data, fail, or execute on an unintended device. A transfer can still be expensive, but it cannot be invisible if it changes the tensor's device type.

Location will eventually need broader treatment. Memory layout, address spaces, remote devices, and accelerator-specific constraints are not part of v0.1. The first experiment asks only whether simple device facts can be checked consistently across the host pipeline.

Ownership

Ownership answers who is responsible for a value and when its resources are released. Core v1 uses single ownership, move semantics, and borrowing. Values that own resources are moved by default rather than implicitly copied. Shared references permit observation; mutable references permit mutation under exclusive access rules.

For deployment code, ownership is more than a memory-safety feature. It documents lifecycle. A decoded request can be consumed by refinement. A tensor can be borrowed by a model call without transferring ownership. A response can take ownership of an output. Cleanup follows scope and drop order rather than a hidden garbage-collection schedule.

The tensor extension deliberately models `Tensor` as an owned, move-only type. Operations borrow operands and return new owned tensors. Implementations may share storage internally,

but observable behavior must look like a fresh result. This design avoids making ordinary tensor operations participate in complicated borrow-carrying relationships.

Constraints

Constraints are the relationships that make an operation valid. Shape equality is one example. Trait bounds, mutability rules, exhaustive matching, model-port compatibility, and artifact-signature verification are others.

A useful diagnostic must do more than announce that a constraint failed. For tensor operations, the extension requires diagnostics to name the operation, state the unsatisfied relationship, and trace symbolic dimensions to their origins. Where a repair is mechanical, the diagnostic should suggest it.

The quality of these messages is part of the product. A theoretically strong type system with inscrutable errors merely moves debugging earlier without making it easier. STARK's implementation plan therefore treats structured source spans and diagnostics as foundational compiler work, not post-release polish.

1.4 Establishing a checked boundary

Static guarantees need a place to begin. Network bytes do not arrive with compiler-verified dimensions. An image decoder can return a dynamic value, but the compiler cannot know which request will be received next Tuesday. An ONNX artifact can be inspected, but its contents are external to the source file and may change independently.

STARK handles this with an explicit boundary between **dynamic evidence** and **static reasoning**.

In the tensor extension, values arriving from outside the type system use erased forms:

STARK

```
// Tensor extension v0.1
TensorDyn<T>    // Element type known; rank and shape dynamic
TensorAny      // Element type, rank, and shape dynamic
```

The operation `refine` consumes such a value, checks runtime evidence once, and returns a statically typed tensor on success:

STARK

```
// Tensor extension v0.1 - illustrative boundary
fn prepare(request: TensorAny) -> Result<Unit, ShapeError> {
  let images = request.refine::<UInt8, [B, 224, 224, 3]>()?;
  inspect_batch(&images);
  Ok(())
}
```

The literal dimensions must match the runtime value. An unbound symbolic dimension such as `B` is bound existentially to the observed size and remains consistent in the enclosing scope. After refinement succeeds, later code reasons about `[B, 224, 224, 3]`; it does not repeatedly ask the tensor for its shape and hope that each branch interprets the answer consistently.



Figure 1-3. Runtime evidence is checked once, then preserved as static type information.

This pattern has three useful properties.

First, uncertainty is visible. A dynamic value does not masquerade as a statically verified one. The program shows where external claims become trusted facts.

Second, failure is localized. If the request contains grayscale pixels, an unexpected rank, or the wrong spatial dimensions, refinement returns an error at the boundary. Downstream functions do not each need defensive shape checks.

Third, the conversion consumes the dynamic value. The program cannot accidentally retain one unchecked alias and one checked alias to the same conceptual input without making that choice explicit.

Refinement does not prove semantic meaning. Successfully refining `[B, 224, 224, 3]` establishes sizes and element type, not that the last axis is RGB or that values are in a particular range. Those concerns require additional types or validation. The distinction keeps the guarantee honest.

A useful rule of thumb

Validate facts at the narrowest boundary that has enough evidence. Preserve validated facts for as long as they remain true. Revalidate only when an operation genuinely erases or changes them.

1.5 Core first, tensor extension second

The earliest STARK concept attempted to be AI-native, cloud-first, actor-based, deployment-oriented, and broadly integrated from the beginning. The current project deliberately narrowed that scope. Before a tensor-aware language can make credible claims about model deployment, it needs ordinary language foundations: a grammar, predictable typing, ownership, errors, modules, collections, and an implementation that agrees with its specification.

The result is a layered design.

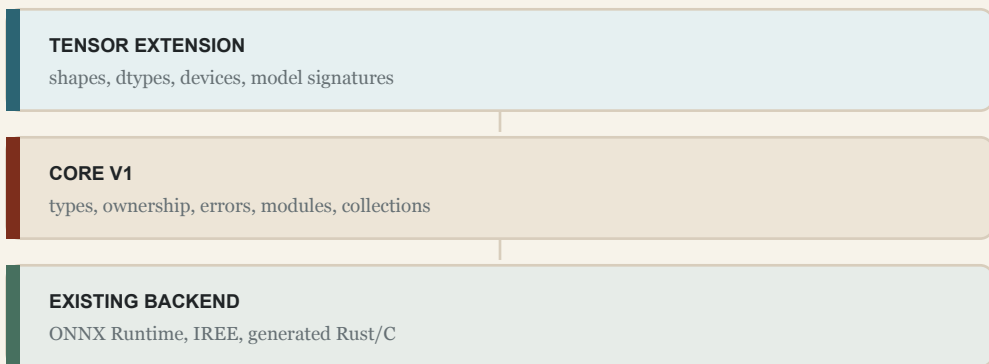


Figure 1-4. Core, extension, and backend have separate responsibilities.

Core v1

Core v1 is a safe, compiled, general-purpose language definition. Its responsibilities include:

- lexical and syntactic grammar;
- primitive and composite types;
- structs, enums, functions, patterns, and control flow;
- local type inference with explicit function signatures;
- generics, traits, and associated types;

- `Option` and `Result` with the `?` operator;
- ownership, moves, borrowing, and deterministic cleanup;
- modules, visibility, imports, packages, and dependency rules; and
- a minimal standard-library surface.

None of those features requires a tensor. A conforming Core v1 implementation must be testable without knowing that the tensor extension exists.

This separation is architectural, not cosmetic. If tensor-specific concepts leak into the core checker's fundamental type representation, every future domain feature risks making the general language less coherent. The implementation plan instead calls for extension-owned type constructors registered behind an explicit flag. Core-only programs must retain Core behavior whether or not an implementation supports tensors.

Tensor extension v0.1

The tensor extension adds the minimum language deltas needed for the deployment experiment:

- `Dim` and `DType` kinds for generic parameters;
- shape expressions as generic arguments;
- `Float16` and `BFloat16` tensor element types;
- an owned `Tensor<T, Shape, device = D>` type;
- dynamic tensor forms and refinement;
- statically typed tensor operations;
- `model` declarations and generated prediction interfaces;
- artifact-signature verification; and
- explicit device placement and transfer.

Training, automatic differentiation, a kernel language, distributed execution, general symbolic inequalities, named axes, and quantization arithmetic are outside v0.1.

Backend integration

The extension defines front-end semantics rather than kernels or a virtual machine. A future implementation may lower checked code to ONNX Runtime host glue, StableHLO and IREE, generated Rust or C, or another existing backend. The compiler's responsibility is to preserve the language's guarantees across that lowering boundary and document any deviations.

The three layers answer different questions:

- **Core:** Is this a well-typed, memory-safe STARK program?
- **Tensor extension:** Are tensor operations and model calls structurally compatible?

- **Backend:** How is the accepted computation executed on real hardware?

Conflating them would make it difficult to tell whether a failure belongs to language semantics, extension semantics, or backend integration.

1.6 A model signature as a type

An inference program typically learns a model's contract through runtime metadata, handwritten wrapper code, generated bindings, or conventions. STARK's tensor extension makes the contract a nominal language item.

STARK

```
// Tensor extension v0.1
model ResNet50<B: Dim> {
  input  image: Tensor<Float32, [B, 3, 224, 224]>;
  output class: Tensor<Float32, [B, 1000]>;
}
```

This declaration introduces a model type. Conceptually, it provides a `load` function that verifies an external artifact and a `predict` method whose input and output types follow the declared ports.

STARK

```
// Generated interface, abbreviated
impl ResNet50 {
  fn load(path: &str) -> Result<ResNet50, ModelError>;

  fn predict<B: Dim>(&self,
    image: &Tensor<Float32, [B, 3, 224, 224]>
  ) -> Tensor<Float32, [B, 1000]>;
}
```

The symbolic batch dimension is universally quantified per prediction call. One call may use a batch of one and another a batch of eight, provided that the input and resulting output agree on the same `B` within each call.

The artifact cannot be trusted merely because the source declaration compiles. `load` must compare the runtime artifact with the declaration before inference begins. Port count, names, order, element types, and dimensions must agree. A literal dimension in source must equal the artifact's static dimension. A symbolic dimension in source may match only a dynamic artifact dimension; otherwise the declaration would promise flexibility that the artifact does not possess.

This creates two checkpoints:

1. **Compile time:** calls made by source code must satisfy the declared model type.
2. **Load time:** the external artifact must satisfy the declaration before it can produce a model value.

The preferred workflow avoids handwritten declarations. An importer inspects the artifact and generates ordinary STARK source:

TEXT

```
stark import model.onnx --out resnet50.stark
```

The generated file can be reviewed and versioned. If the artifact changes, regeneration produces a visible source diff. If source and artifact drift apart, loading fails before the first inference call.

Illustrative workflow

`stark import` is specified as recommended tooling and appears in the active implementation roadmap. It is not implemented in the current repository.

1.7 Following one image through the type system

The following example shows the intended relationship between refinement, preprocessing, and a model call. It is deliberately small; later chapters will examine each operation and rule in detail.

STARK

```
// Tensor extension v0.1 - illustrative end-to-end fragment
use tensor::*;

model ResNet50<B: Dim> {
  input  image: Tensor<Float32, [B, 3, 224, 224]>;
  output class: Tensor<Float32, [B, 1000]>;
}

fn classify<B: Dim>(<
  model: &ResNet50,
  images: &Tensor<UInt8, [B, 224, 224, 3]>
) -> Tensor<Float32, [B, 1000]> {
  let normalized = images
    .permute::<[0, 3, 1, 2]>()
    .cast::<Float32>()
    .div(&IMAGENET_STD)
    .sub(&IMAGENET_MEAN);

  model.predict(&normalized)
}

fn main() -> Result<Unit, ServeError> {
  let model = ResNet50::load("resnet50.onnx")?;
  let raw: TensorAny = read_request()?;
  let images = raw.refine::<UInt8, [B, 224, 224, 3]>()?;
  let logits = classify(&model, &images);
  respond(&logits)
}
```

Begin at `read_request`. The request is external, so its dtype and shape are not trusted. It produces `TensorAny` inside a `Result`. The first `?` propagates a request-level failure.

Next, `refine` checks that the runtime value contains `UInt8` samples, has rank four, has spatial dimensions `224` by `224`, and has three values on the final axis. It binds `B` to the observed batch size. The second `?` propagates a `ShapeError` through the surrounding error type.

The function `classify` accepts only NHWC-shaped unsigned bytes. It borrows the images, so the caller retains ownership. `permute` transforms the shape from `[B, 224, 224, 3]` to `[B, 3, 224, 224]`. `cast` makes the element type `Float32`. Normalization constants must satisfy the extension's statically provable broadcasting rule. The result passed to `predict` must be exactly compatible with the model port.

If `permute` is removed, the model call receives `[B, 224, 224, 3]` where `[B, 3, 224, 224]` is required. The inner dimensions are not interchangeable merely because both shapes have four axes.

TEXT

```
error[E3104]: incompatible model input `image`
  required: Tensor<Float32, [B, 3, 224, 224]>
     found: Tensor<Float32, [B, 224, 224, 3]>
  note: dimension 1 originates from `images` axis 1 (224)
  help: channel-last input may require `.permute::<[0, 3, 1, 2]>()`
```

This diagnostic is illustrative rather than an implemented compiler message, but it demonstrates the required product behavior: identify the model port, show both types, trace the mismatch, and suggest a mechanical repair when the likely intent is clear.

If `cast` is removed, shape compatibility is not enough. The model expects `Float32`, while preprocessing still produces `UInt8`. STARK defines no implicit dtype conversion. The program must say where and how the representation changes.

If a normalization constant has an incompatible shape, the elementwise operation fails before the model call. If the model artifact changes to accept a different spatial size, `load` fails before inference. If the tensor and model are placed on incompatible devices, their types do not unify.

None of this proves that the model is accurate, that the normalization constants are numerically correct, or that the chosen labels are meaningful. Those remain matters for tests, artifact governance, and domain validation. The type system targets structural contradictions, not all possible mistakes.

1.8 Predictability in the general-purpose core

Tensor types are the project's differentiating experiment, but they sit on general language rules chosen for predictability.

Explicit numeric behavior

Core v1 does not perform implicit numeric conversions. A conversion that may change representation appears in source. Integer overflow traps consistently rather than changing between debug and optimized builds. Indexing is bounds checked. Safe lookup APIs return `Option` instead of manufacturing a sentinel.

These choices favor a stable mental model. A deployment program should not acquire different arithmetic behavior because a build profile changed, nor should a signed value silently become unsigned at a call boundary.

Exhaustive alternatives

Enums and exhaustive `match` expressions make states explicit. A result is not an untyped exception channel; it is `Result<T, E>`. Absence is not a null pointer accepted by every reference type; it is `Option<T>` where absence is part of the interface.

STARK

```
// Core v1
enum LoadState {
    Ready,
    Missing,
    Incompatible(String)
}

fn describe(state: LoadState) -> String {
    match state {
        LoadState::Ready => "ready".to_string(),
        LoadState::Missing => "artifact missing".to_string(),
        LoadState::Incompatible(reason) => reason
    }
}
```

Adding a new variant forces relevant matches to be reconsidered. That does not guarantee a good policy, but it prevents a state from disappearing into an implicit default.

Ownership without garbage collection

Core v1 aims for memory safety without a tracing garbage collector. A value has one owner. Assignment and parameter passing move non-`Copy` values unless the program borrows them. Immutable borrows may be shared; a mutable borrow is exclusive. Values are dropped automatically when their owners leave scope.

STARK

```
// Core v1
fn inspect(name: &String) -> UInt64 {
    name.len()
}

fn consume(name: String) {
    println(name.as_str());
} // name is dropped here

fn main() {
    let model_name = "resnet50".to_string();
    let count = inspect(&model_name); // borrowed, still owned by main
    consume(model_name);              // moved into consume
    // Using model_name here would be an error.
}
```

The ownership system is conservative and intentionally simpler than Rust's full lifetime system. Core v1 uses annotation-free lexical lifetime rules. This may reject some safe programs that a more sophisticated checker could accept. The trade-off is an implementation target small enough to build and explain.

Explicit function boundaries

Local values may be inferred, but function signatures are explicit. Public contracts do not depend on inference across arbitrary module boundaries. Generics carry trait bounds. Modules and visibility determine which names form the supported interface.

Together, these rules establish the environment in which tensor contracts can be trusted. A perfect shape checker would be of limited value if ordinary resource lifetimes, errors, conversions, and module boundaries remained ambiguous.

1.9 What STARK deliberately does not promise

New language projects are especially vulnerable to aspirational documentation. Features are described in present tense, architecture diagrams imply implemented subsystems, and performance targets become quoted as measured results. STARK's current documentation separates normative drafts, active plans, optional extensions, and archived historical designs to avoid that failure.

At this stage, STARK does not promise:

- a working compiler or runnable standard library;
- faster inference than Python, C++, Rust, or any framework;
- a new tensor kernel runtime or graph compiler;
- training, automatic differentiation, or optimizer APIs;
- automatic conversion of arbitrary Python programs;
- complete semantic typing for images, audio, or robotics;
- cloud deployment syntax or a package registry;
- distributed execution, actor systems, or serverless primitives in Core v1;
- hard real-time execution or worst-case timing guarantees; or
- that a new language will ultimately be the best packaging for these ideas.

Some of these appeared in the project's broader pre-pivot vision. They remain historical context, not current commitments.

The absence of a performance claim is particularly important. Removing Python orchestration does not automatically make model kernels faster; those kernels already execute in optimized native backends in many systems. A fair evaluation must control the backend, workload, preprocessing, warm-up, concurrency, and measurement method. The active roadmap records artifact size, startup time, peak memory, and steady-state latency during the prototype, but performance is not a gate until a controlled baseline exists.

Credibility rule

A specification defines intended behavior. An implementation demonstrates that the behavior can be built. A benchmark measures one implementation under stated conditions. None of these substitutes for the others.

1.10 The implementation path is part of the argument

STARK's roadmap is gate-based rather than date-based. Each gate asks for evidence needed by the next.

Gate 1: Core front end

A lexer and parser must accept the normative grammar, produce useful source locations, and classify the specification's fixtures deterministically. Grammar ambiguities discovered during implementation must be fixed in the normative source rather than hidden in compiler behavior.

Gate 2: Core semantic checker

Name resolution, type checking, inference, move analysis, and borrow checking must work on representative Core programs. Tensor-specific rules must remain outside the Core checker.

Gate 3: Minimal execution path

Enough Core STARK must execute to support the deployment experiment. The plan currently favors a tree-walking interpreter because it is the shortest path to observable semantics. A custom VM is not required.

Gate 4: Tensor front end and ONNX import

The tensor extension is added behind an explicit feature boundary. The compiler must reason about dimensions, operations, devices, model declarations, generated signatures, and artifact verification for at least one representative ONNX model.

Gate 5: Deployment prototype

One realistic computer-vision program must cover refinement, preprocessing, model invocation, and postprocessing while using an existing backend. A defect corpus must demonstrate incompatible dimensions, incorrect element types, device mismatches, and artifact-signature drift. Measurements are reported reproducibly rather than marketed selectively.

Gate 6: Decision checkpoint

The project records a go, revise, or stop decision. It proceeds only if the safety or deployment advantage is material and not erased by integration complexity. A successful result may justify a narrower follow-up experiment in semantic image annotations. An unsuccessful result may show that a library, schema generator, or existing compiler extension is the better vehicle.

This willingness to stop is not a lack of ambition. It is a design constraint. The goal is not to prove that every initial idea deserves implementation. The goal is to discover whether the central idea survives contact with a real compiler, a real artifact, and a real deployment program.

1.11 A practical definition of success

For STARK's first experiment, success is not broad adoption, a large package ecosystem, or benchmark dominance. It is a smaller, observable outcome.

A valid pipeline should:

- import a representative model signature rather than relying on a handwritten guess;
- refine external request data at an explicit boundary;
- preserve symbolic batch information through preprocessing;
- reject incompatible shapes, element types, and devices before execution;
- detect artifact/declaration drift before inference;
- produce a deployable native artifact through an existing backend;
- agree numerically with a reference implementation within a documented tolerance; and
- remain small enough that its types and diagnostics can be explained in a focused demonstration.

The decisive comparison is not "Does STARK look elegant?" It is "Does STARK reveal important defects earlier and more clearly than the alternatives, at an acceptable implementation and integration cost?"

That question protects the project from two opposite errors. The first is premature dismissal: assuming that ordinary host-language wrappers are necessarily sufficient without testing an

end-to-end typed pipeline. The second is premature triumph: assuming that moving dimensions into types automatically produces a usable language.

A new language pays a high adoption cost. It needs tooling, documentation, interoperability, debugging, stable semantics, package management, and a contributor community. Static tensor checks must deliver enough value to justify that cost. The prototype is designed to make that trade-off visible.

1.12 Reading the rest of this book

This book uses four status labels so that syntax and implementation maturity are never ambiguous:

- **Core v1** identifies behavior defined by the normative core specification.
- **Tensor extension v0.1** identifies behavior defined by the optional tensor and model specification.
- **Illustrative** identifies planned commands, diagnostics, or integration code that explains intent but is not yet implemented.
- **Historical** identifies superseded ideas included to explain how the design evolved.

The next chapters begin with ordinary Core programs: values, functions, control flow, structs, enums, patterns, errors, ownership, and traits. Only after that foundation is established does the book introduce tensor kinds, symbolic dimensions, model declarations, and ONNX import.

That order reflects the architecture. STARK is not a tensor DSL with an accidental host language attached. It is an attempt to build a small general-purpose core on which a deployment-focused tensor extension can make stronger guarantees.

Whether that attempt succeeds remains an engineering question. The specifications provide a testable hypothesis. The compiler and prototype must provide the evidence.

Chapter summary

- STARK targets the deployment program surrounding a model, not a replacement for mature numerical backends.
- Its review lens covers shape, meaning, location, ownership, and constraints.
- Core v1 defines a safe general-purpose foundation with static types, explicit semantics, ownership, borrowing, `Option`, and `Result`.
- Tensor extension v0.1 adds symbolic shapes, element types, devices, model signatures, refinement, and artifact verification.
- Dynamic values become statically useful through an explicit, fallible refinement boundary.

- Model compatibility is checked twice: source calls at compile time and external artifacts at load time.
- The project deliberately avoids unmeasured performance claims and broad pre-prototype scope.
- A gate-based roadmap culminates in a go, revise, or stop decision based on a real computer-vision pipeline.

Questions for reflection

1. In a deployment system you know, where are tensor layout and dtype assumptions recorded today? Which of them are mechanically checked?
2. Which failures require runtime evidence, and which are contradictions already present in source declarations?
3. What information does a successful shape refinement establish? What important semantic information does it not establish?
4. Why does the tensor extension require a model artifact to be verified even when all source code type-checks?
5. What value would STARK need to demonstrate to justify the cost of adopting a new language instead of a library or code generator?

Source notes

1. The current project status and Core/extension distinction follow [README.md](#) and [STARKLANG/docs/index.md](#).
2. Core principles and maturity language follow [STARKLANG/docs/spec/00-Core-Language-Overview.md](#).
3. Tensor shapes, refinement, devices, model declarations, diagnostics, and backend boundaries follow [STARKLANG/docs/extensions/Tensor-Model-Types.md](#), extension `tensor v0.1`.
4. The five-concern review lens and deployment wedge follow [STARKLANG/docs/ROADMAP.md](#).
5. Compiler architecture and gate implementation choices follow [STARKLANG/docs/PLAN.md](#).
6. The original broad scope and subsequent core-first pivot are described in [STARK_Analysis_and_Discussion.md](#); archived implementation and performance claims are not treated as current facts.